Generating Class Diagrams from Structured Use Case Descriptions with LLMs

Evin Aslan Oğuz[a], Jochen M. Küster[b] and Felix Lennart Schildmann[c]
Bielefeld University of Applied Sciences, Interaktion 1, Bielefeld, 33619, Germany

Keywords: Large Language Models (LLMs), Use Case Descriptions, Domain Class Diagrams, Class Diagram Generation.

Abstract: This paper explores the use of large language models (LLMs) for automating the generation of domain class diagrams from structured use case descriptions. While existing research projects have applied LLMs to generate domain class diagrams using unstructured input, this work introduces another approach that leverages standardized use case description forms and OpenAI's structured outputs feature to improve the generated class diagrams. The proposed approach generates PlantUML code that can be directly visualized as domain class diagram. The approach is quantitatively evaluated using a small set of structured use case descriptions, with the resulting domain class diagrams. The system achieved a macro-average F1 score of 0.91, demonstrating strong overall performance, although challenges remain, particularly in modeling relationships, where accuracy was significantly lower. The results highlight the potential of LLMs supporting early stage software modeling and provide a foundation for future improvements, though challenges remain in correctly inferring relationship multiplicities in more complex scenarios.

1 INTRODUCTION

Artificial intelligence (AI) and natural language processing (NLP) technologies have increasingly transformed the field of software engineering. Recent advancements in large language models (LLMs) have enabled automation in diverse software engineering tasks, including code generation, defect detection, and documentation support (Fan et al., 2023; Kokol, 2024). These technologies are reshaping how developers design, implement, and maintain software systems, offering opportunities for improved productivity and quality. Within this trend, AI-assisted development tools have gained significant momentum, particularly in later phases of the software development life cycle, such as implementation and testing (Li et al., 2022; Wang et al., 2024b).

Despite these advancements, the application of AI and NLP in the early stages of software development remains relatively limited. One of the essential artifacts derived in the early stages of software development is the class diagrams. They provide a structured representation of system architecture that supports clear communication among stakeholders and ensure design consistency. They help identify relationships, attributes, and potential design flaws early in development, improving understanding, maintainability, and overall software quality (De Bari et al., 2024). However, the creation of class diagrams is still largely performed manually. This manual process is time consuming, cognitively demanding, and prone to human error, especially in complex systems.

While several studies have explored automatic class diagram generation from natural language requirements text (Babaalla et al., 2025; Ullrich et al., 2025; De Bari et al., 2024; Anda and Sjøberg, 2005), user stories (Li et al., 2024; Arulmohan et al., 2023), or source code (Shehata et al., 2024), there is still no known attempt to generate class diagrams directly from structured use case descriptions. Use case descriptions offer several advantages as input for automatic class diagram generation as they are typically written in a semi-structured format, following standardized templates such as the one proposed by Cockburn (Cockburn, 1998), which include elements such as actors, preconditions, postconditions, main scenarios, and extensions.

Considering that the manual derivation of class diagrams is time consuming, cognitively demanding, and prone to human error, and given the advantages

[a] https://orcid.org/0000-0002-2750-7667
[b] https://orcid.org/0009-0004-0271-6069
[c] https://orcid.org/0009-0004-8516-4166

Aslan Oğuz, E., Küster, J. M. and Schildmann, F. L.
Generating Class Diagrams from Structured Use Case Descriptions with LLMs.
DOI: 10.5220/0014402300004058

In *Proceedings of the 14th International Conference on Model-Based Software and Systems Engineering (MODELSWARD 2026)*, pages 350-361
ISBN: 978-989-758-798-6; ISSN: 2184-4348

of using use case descriptions for automatic class diagram generation, this paper proposes an approach on how large language models (LLMs) can be employed to automate this process. To evaluate the effectiveness of this approach, the following research questions are addressed.

RQ1: Can LLMs generate domain class diagrams consisting of classes, attributes, and relationships from structured use case descriptions?

RQ2: How complete are generated domain class diagrams compared to manually designed domain class diagrams?

To answer these question, a set of use case descriptions is processed using a selected LLM. The resulting class diagrams are compared to manually designed class diagrams and evaluated in terms of their accuracy, quality, and recurring error patterns.

The remainder of this paper is organized as follows. Section 2 provides background information on use case descriptions and class diagrams. Section 3 reviews the related work. The proposed approach is described in detail in Section 4, followed by the evaluation in Section 5. Finally, Section 6 concludes the paper and outlines future work.

2 BACKGROUND

2.1 Use Case Descriptions

A use case formally models a specific goal or objective that a user wants to achieve through interaction with a system. This interaction sequence captures system requirements by defining the steps necessary to achieve the user's goal (Merrick and Barrow, 2005). The use case descriptions set an important foundation during the requirements analysis, which is an early stage of the entire software development process. Based on (Cockburn, 1998), the structure of a use case can be described by a table with the following rows: Name, Primary User, Goal, Precondition, Postcondition, Additional Users, Standard Procedure and Extensions. This structure can be seen in an example use case description for an employee creating an order in an ERP system in Table 1.

Table 1: Example of used benchmarking use case.

Name	Create Order
Primary User	Employee
Goal	To create a new order in the ERP system
Precondition	The employee is logged in the system
Postcondition	1. The new order was saved in the ERP System 2. A success message was displayed to the employee
Additional Users	None
Standard Procedure	1. The employee navigates to the 'Orders' page 2. The system shows a page containing a list of all orders and a button to create new orders 3. The employee clicks on the button for order creation 4. The system shows a form with fields for the customer and for one or more order positions with product, quantity and price each 5. The employee selects a customer by name or customer id, one or more products, sets quantity and price(s) and submits the form 6. The system calculates the total for the order, generates an order ID, saves the order in the database of the ERP system and displays a success message
Extensions	

2.2 Class Diagrams

Class diagrams are visual representations of a software systems structure in terms of classes and their relationships; in which classes are abstractions that specify attributes and behavior of the correlated set of objects (Bruegge and Dutoit, 2013, p. 48). In the UML they are depicted by boxes that are separated into three segments; the top segment contains the name of the class, the center segment its attributes, and the bottom segment its operations (Bruegge and Dutoit, 2013, p. 48).

Classes can have several types of relationships between each other; a prominent example for these are associations. Associations between classes represent a group of links between corresponding objects; they can be labeled with roles and multiplicities for a better distinction and higher value of information on the solution domain (Bruegge and Dutoit, 2013, pp. 50-52). Another important relationship is inheritance; it describes the relationship between a general and one or more specialized classes (Bruegge and Dutoit, 2013, p. 55).

Class diagrams in general can be divided into domain class diagrams and design class diagrams (Larman, 2004, pp. 8-10). In this paper we are focusing on domain class diagrams.

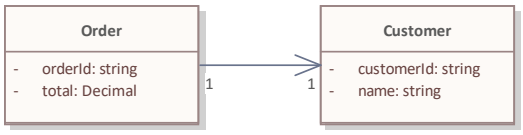

Figure 1: An example domain class diagram based on Create Order use case.

A domain class diagram describes simple domain objects or conceptual classes along with their key at-

tributes and relationships between them, without including any methods or implementation details (Larman, 2004, p. 134). In the software development process, they are created after the definition of use cases (Larman, 2004, pp. 8-10). Figure 1 shows an example of a simple domain class diagram Order and Customer as conceptual classes along with relationships with multiplicities. They contain only simple attributes and no methods.

2.3 PlantUML

PlantUML[1] is an open-source tool that allows users to create Unified Modeling Language (UML) diagrams from plain text descriptions (PlantUML Team, 2025b). It supports various UML diagram types, including class, sequence, and use case diagrams, and is widely used for documentation and model visualization in software engineering.

To generate a UML class diagram in PlantUML, the diagram must be defined within *@startuml* and *@enduml* statements. Inside this block, classes, their attributes and methods (with visibilities), as well as relationships, are specified using a simple text-based syntax (PlantUML Team, 2025a). An example illustrating two classes with inheritance is shown below:

```
@startuml
title Example
class Person {
  - name: String
  + getName(): String
}
class Student {
  + studentNumber: String
}
Person --|> Student
@enduml
```

Code Listing 1: An example class diagram in PlantUML.

This example defines two classes, `Person` and `Student`, where `Student` inherits from `Person`. Attribute and method visibilities are denoted by symbols: `+` (public), `-` (private), `#` (protected), and `~` (package-private). The inheritance relationship is represented by `--|>`, while other relationships such as associations (`-->`), implementations (`..|>`), compositions (`*--`), and aggregations (`o--`) follow similar notations (PlantUML Team, 2025a).

Figure 2 shows the class diagram produced from the above PlantUML code (see Code listing 1).

[1]https://plantuml.com/

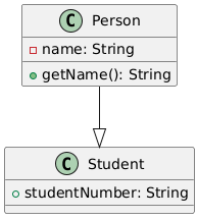

Figure 2: Generated class diagram of the example PlantUML code in Code listing 1.

3 RELATED WORK

The integration of artificial intelligence (AI) and large language models (LLMs) into software engineering has attracted increasing research attention. Recent studies highlight the rapid growth of AI-related publications in this domain, with natural language processing (NLP) emerging as a key enabler for automating software development tasks (Kokol, 2024). Several works have explored how LLMs can assist in model-driven engineering, including the generation of UML artifacts from textual inputs (Cámara et al., 2023; Wang et al., 2024a; Ferrari et al., 2024).

Research on class diagram generation has expanded in recent years. Studies have shown that LLMs can create UML class diagrams from requirements text or user stories, achieving syntactic correctness but limited semantic completeness (De Bari et al., 2024; Li et al., 2024). Other work has investigated generation from source code with LLMs (Shehata et al., 2024). Additional research has focused on domain model extraction and iterative refinement methods to improve model completeness and accuracy (Arulmohan et al., 2023; Chen et al., 2023; Yang et al., 2024). From a model-driven perspective, end-to-end pipelines connecting textual requirements to UML models and source code have also been proposed (Babaalla et al., 2025).

Earlier approaches demonstrated that use cases can effectively support class diagram derivation, although automation remained limited and error prone (Anda and Sjøberg, 2005; Shweta et al., 2021). More recent findings emphasize the importance of structured input representations to enhance LLM accuracy in requirements analysis (Ullrich et al., 2025).

Building on these insights, this paper extends the authors bachelor thesis Schildmann (2025), which explored the feasibility of generating class diagrams from structured use case descriptions using LLMs. In contrast to existing studies that primarily focus on unstructured texts, user stories, or source code, this paper emphasizes the potential of use case descriptions as an information-rich input source, including actors, scenarios, and extensions.

4 OVERVIEW OF THE APPROACH

The proposed approach aims to automate the generation of UML class diagrams from structured use case descriptions using large language models (LLMs). Unlike previous studies that rely on unstructured requirements, user stories, or source code, this work leverages use case descriptions written in a standardized, structured format. Such descriptions typically follow a well established template that includes fields like use case name, actors, preconditions, postconditions, main scenario steps, and extensions. These fields capture both the functional aspects of a system and its interaction context, offering a rich and less ambiguous foundation for class diagram derivation.

The overall workflow of the proposed approach consists of three main phases: (Step 1) preprocessing of structured use case descriptions, (Step 2) structured output generation through prompt driven interaction with an LLM, and (Step 3) postprocessing consisting of generating PlantUML code and rendering the class diagram.

The entire process is visualized in Figure 3, highlighting the activities and decision points attributed to the user, the application, and the LLM.

First, the use case descriptions are uploaded to the application. The application then parses the uploaded files (Step 1) and generates a prompt with explicit instructions for creating a domain class diagram (Step 2.1). Next, the LLM is prompted together with the use case descriptions (Step 2.2) and produces the class diagram data in a structured output format, which is returned to the application. The application converts this data into PlantUML code and renders the corresponding diagram (Step 3). Finally, both the generated PlantUML code and the rendered diagram are presented to the user. The following subsections describe each phase in more detail.

4.1 Preprocessing of Use Case Descriptions

In the first phase, the input use case descriptions are collected in a predefined form or in Excel format to ensure consistency across samples. The user can either fill out a predefined form that follows the Cockburn (Cockburn, 1998) template manually or upload an Excel file containing one or multiple use case descriptions. The predefined form is designed to guide the user through each key field of the template—such as Name, Primary Actors, Preconditions, Postconditions, Main Scenario, and Extensions ensuring that all required information is provided in a clear and standardized manner. When using the Excel based option, the system automatically reads each worksheet and row as a separate use case entry, enabling batch processing of multiple use cases at once. During this phase, each description undergoes a structural validation process that checks for the presence and completeness of all mandatory fields. This validation ensures that the input conforms to the expected Cockburn style schema, thereby minimizing inconsistencies and reducing the likelihood of misinterpretation by the language model during the generation phase.

4.2 Generating an LLM Prompt and LLM Response

In the second phase, the core transformation process takes place via OpenAIs GPT-4.1 model. The system constructs a prompt that includes the use case fields and instructions guiding the model to extract entities, their attributes, and potential associations implied by the narrative. For example, actors and nouns in the preconditions or main flow often map to candidate classes, while actions and relationships expressed through verbs suggest associations or methods.

Code Listing 2 shows the communication with the LLM: First, the base system prompt is defined; it contains instructions on the expected input and output, and notes to prevent common mistakes that were discovered in early test runs of the project, e.g. listing the generic 'system' actor as class. After that, the specialization prompts follow; these include additional, more precise instructions and output requirements for domain class diagrams, such as they should include classes, attributes, and relationships, without further implementation details.

The *generate()* method serves as the core component of the UML generation process. It takes a list of structured use case objects as input and converts them into a suitable format for large language model (LLM) prompting. First, the predefined system prompt, which defines the task and constraints, is combined with the concatenated use case descriptions to form the complete input message.

Using the configured OpenAI client, a structured API call is made via the *chat.completions.parse()* method, which sends both the system and user messages to the LLM. The *response_format* parameter ensures that the output is parsed directly into a predefined *UMLDiagram* object, enabling reliable extraction of classes, attributes, and relationships.

Finally, the resulting *UMLDiagram* instance is passed to the *PlantUMLGenerator.convert_uml_object_to_plantuml()* method, which

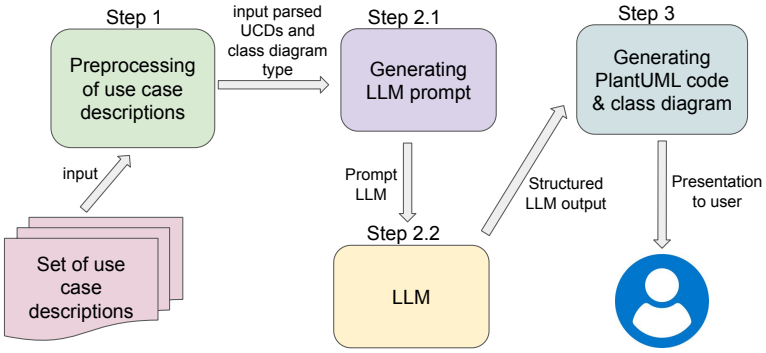

Figure 3: Overview of the approach.

converts the structured diagram into a textual PlantUML representation. The details of this method is explained in the next Section (see Section 4.3).

An example generated LLM prompt based on the Create Order use case description (see Table 1, to generate a domain class diagram, is shown in Code Listing 3. The code listing illustrates the complete prompt sent to the large language model (LLM). The prompt consists of two structured messages formatted as a JSON array, representing the typical input to OpenAIs chat based API.

The first element, labeled with the ”role”: ”system”, defines the system level instructions that guide the LLMs behavior and output constraints. This section contains detailed guidance on the expected task of transforming one or more use case descriptions into a UML class diagram. It explicitly instructs the model to exclude implementation specific elements such as visibility modifiers, methods, or complex logic, and to focus solely on domain relevant classes, attributes, and relationships. It also specifies the notation style for multiplicities (e.g., 0..1, 1.., etc.) and clarifies that the System actor mentioned in the use case should not be modeled as a class.

The second element, labeled with the ”role”: ”user”, provides the actual Create Order use case description in a structured text format following the Cockburn template. It includes all key sections such as Name, User, Goal, Preconditions, Postconditions, Standard Procedure, and Extensions. This format ensures that the LLM receives a consistent and semantically rich representation of the scenario, enabling it to infer relevant domain concepts such as Order, Customer, and Product, as well as their relationships.

The response generated by the LLM is structured in a predefined object-oriented format to ensure consistency and facilitate automated processing in subsequent steps. Instead of producing unstructured natural language output, the model returns a serialized representation of a UMLDiagram object, which encapsulates the entire domain class diagram. The structured response includes a list of UMLClass objects—each containing class names, attributes with types and visibility, and placeholders for methods—as well as UMLRelationship objects that define associations and compositions between classes with explicit multiplicities. Inclusion of attributes such as type, source, target, and multiplicity allows direct transformation into PlantUML code or other visual UML formats without additional manual intervention.

4.3 Generating PlantUML Code and Class Diagram Visualization

In the third phase, the LLM output that is produced in second step (see Section 4.2) is processed to be in PlantUML code and then it is visualized.

Since the default output of OpenAI is a string, and complex objects are required, another feature called ’structured outputs’[2] had to be used. Structured outputs enforce the output to follow a predefined schema, for example, a particular class, and enumerations have been used wherever possible to constrain model outputs to a fixed set of predefined values.

In order to generate a semantically correct output that can be rendered into a diagram image, the LLM will not be prompted to generate PlantUML code directly, but objects of classes that represent artifacts of a class diagram. The class structure covers all necessary information to generate a UML class diagram from; this includes classes, enumerations, attributes, visibilities, and relationships.

The *convert_uml_object_to_plantuml()* method performs the transformation of a structured *UMLDiagram* object into a valid PlantUML text representation. It begins by initializing a list of lines with the PlantUML start directive *@startuml* and

[2]https://platform.openai.com/docs/guides/structured-outputs

```python
import openai
from core.UseCase import UseCase
from core.Config import Config
from core.uml import PlantUMLGenerator
from core.uml.UMLAIClasses import UMLDiagram

class UMLGenerator:

    def __init__(self):
        self.config = Config()
        self.model = self.config.get("openai", "model")
        self.client = openai.OpenAI(api_key=self.config.get("openai", "key"))
        self.sys_prompt = """
        You will be given one or more use case descriptions for a software system.
        Your task is to generate a UML diagram that represents resulting classes
        and their properties based on these descriptions.
        Note that the 'System' actor described is not a class
        but rather the whole system itself.
        Use the following multiplicity notation: 0..1, 1, 0..*, 1..*, etc.

        The UML diagram should only include simple domain classes,
        their attributes and relationships.
        Do not include any implementation details, visibility,
        relationship roles, methods, or complex logic.
        The diagram should be simple and focused on the domain model."""

    def generate(self, use_cases: list[UseCase]) -> str:
        prompt = self.sys_prompt
        use_case_descriptions = "\n\n".join(str(use_case) for use_case in use_cases
                                            )
        completion = self.client.beta.chat.completions.parse(
                    temperature=float(self.config.get("plantuml", "temperature")),
                    model=self.model,
                    messages=[
                        {"role": "system", "content": prompt},
                        {"role": "user", "content": use_case_descriptions}
                    ],
                    response_format=UMLDiagram
                )
        response = completion.choices[0].message.parsed
        return PlantUMLGenerator.convert_uml_object_to_plantuml(response)
```

Code Listing 2: UMLGenerator class that generates LLM prompts.

optionally appends a title if provided within the diagram object. Each class contained in the *UMLDiagram* is then converted using the helper function *class_to_plantuml()*, which formats class definitions according to PlantUML syntax. Enumerations are processed in a similar loop, where literals are listed between *enum* brackets.

Afterwards the method iterates through all defined relationships, invoking the *format_relationship()* helper to generate appropriate association, aggregation, or composition, links between classes, including multiplicities and directions. The process concludes with the *@enduml* directive, signaling the end of the PlantUML script. Finally, the accumulated lines are joined into a single string and returned.

An example of generated PlantUML code based on the Create Order use case description (see Table 1) can be seen in code listing 5. It represents the domain class diagram generated by the LLM for the Create Order use case. The diagram models the core entities and relationships of an order management process within an ERP system, derived solely from the structured use case description.

Each class block defines a key domain concept along with its essential attributes. The Order class encapsulates an orderId and a total, representing the unique identifier and the aggregated monetary value of the order. The OrderPosition class captures line-level details through quantity and price attributes, reflecting the individual items within an order. The

```
[
  {
    "role": "system",
    "content": "
        You will be given one or more use case descriptions for a software system.
        Your task is to generate a UML diagram that represents resulting classes
        and their properties based on these descriptions.
        Note that the 'System' actor described is not a class but rather the whole
    system itself.
        Use the following multiplicity notation:
        0..1, 1, 0..*, 1..*, etc.

        The UML diagram should only include simple domain classes, their attributes
     and relationships.
        Do not include any implementation details, visibility, relationship roles,
    methods, or complex logic.
        The diagram should be simple and focused on the domain model.
    "
  },
  {
    "role": "user",
    "content": "*Name:* Create Order\n*User:* Employee\n*Goal:* To create a new
    order in the ERP system\n*Preconditions:*\n1. The employee is logged in the
    system\n*Postconditions:*\n1. The new order was saved in the ERP System\n2. A
    success message was displayed to the employee\n*Additional Users:*\n1. None\n*
    Standard Procedure:*\n1. The employee navigates to the 'Orders' page\n2. The
    system shows a page containing a list of all orders and a button to create new
    orders\n3. The employee clicks on the button for order creation\n4. The system
    shows a form with fields for the customer and for one or more order positions
    with product, quantity and price each\n5. The employee selects a customer by
    name or customer id, one or more products, sets quantity and price(s) and
    submits the form\n6. The system calculates the total for the order, generates
    an order ID, saves the order in the database of the ERP system and displays a
    success message\n*Extensions:*\nNone"
  }
]
```

Code Listing 3: Generated LLM prompt based on the Create Order use case description asking for a domain class diagram.

Product and Customer classes define supporting entities, each with identifiers (productId, customerId) and descriptive attributes (name).

The relationships at the bottom of the code specify cardinalities that reflect the real-world structure of the domain. An Order is composed of one or more OrderPosition elements (aggregation denoted by *–), each linked to exactly one Product. Additionally, each Order is associated with exactly one Customer. These associations are expressed using PlantUMLs textual syntax for UML relationships.

When rendered with PlantUML, this code visually presents a domain class diagram that captures the structural essence of the Create Order process. The generated model confirms that the LLM correctly identified key entities and their interconnections from the textual use case input. Generated domain class diagram of Create User use case description based on the Code listing 5 is shown in Figure 4.

The generated class diagram captures the essential entities and their associations within an ERP order management context, effectively translating the textual use case information into a structured visual representation.

At the center of the model lies the Order class, which represents the main business entity. It contains two attributes—orderId and total—describing the unique identifier and total monetary value of an order. The Order class is associated with one or more instances of OrderPosition, represented by the 1..* multiplicity, indicating that each order can include multiple order lines.

The OrderPosition class describes individual order entries, each specifying a product's quantity and price. It connects to exactly one Product, which is represented by its productId and name attributes. This reflects the dependency of each order line on a specific product.

```
def convert_uml_object_to_plantuml(
    diagram: UMLDiagram) -> str:
    lines = ["@startuml"]

    # Title
    if diagram.title:
        lines.append(f"title {diagram.
    title}")

    # Classes
    for cls in diagram.classes:
        lines.append(class_to_plantuml(
    cls))

    # Enums
    for enum in diagram.enums:
        lines.append(f"enum {enum.name}
     {{")
        for literal in enum.literals:
            lines.append(f"  {literal
    }")
        lines.append("}")

    # Relationships
    for rel in diagram.relationships:
        lines.append(
    format_relationship(rel))

    lines.append("@enduml")
    return "\n".join(lines)
```

Code Listing 4: Method for UML diagram object to PlantUML conversion.

```
@startuml
title ERP Order Domain Model
class Order {
  + orderId: String
  + total: Decimal
}
class OrderPosition {
  + quantity: Integer
  + price: Decimal
}
class Product {
  + productId: String
  + name: String
}
class Customer {
  + customerId: String
  + name: String
}
Order "  1" *-- "  1..*" OrderPosition
Order "  1" --> "  1" Customer
OrderPosition "  1" --> "  1" Product
@enduml
```

Code Listing 5: Generated PlantUML code based on the Create Order use case description.

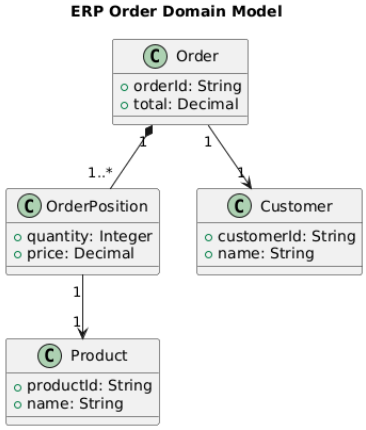

Figure 4: Generated domain class diagram of Create Order use case via PlantUML.

Additionally, each Order is linked to exactly one Customer, modeled through the 1-to-1 association. The Customer class includes customerId and name attributes, defining the customer who places the order.

5 EVALUATION

To illustrate and assess the accuracy of the proposed approach, an evaluation strategy was implemented. The evaluation focuses on analyzing the accuracy of the generated domain class diagram diagrams. Accuracy is measured using standard metrics such as precision, recall, and F1-score, which help determine how well the approach identifies correct classes, attributes, and relationships across different test scenarios.

Therefore the use case descriptions are enriched with the expected classes, attributes and relationships, which is concatenated at the end of use case description after Extensions. Table 2 presents an example use case describing the process of creating an order in an ERP system, along with the expected set of classes, attributes, and relationships. The generated results are compared with the expected elements and classified as true positives, false positives, or false negatives.

Naming variations are accepted as long as the semantic meaning is preserved. For example, an element named OrderPosition is considered correct if Position is the expected name. Relationships are also compared based on structure and multiplicity; however, when multiplicities cannot be clearly determined from the use case description, they are omitted and treated as wildcards.

Missing expected elements are counted as false negatives, while incorrect or additional elements—such as relationships with wrong multiplicities or redundant classes—are treated as false positives. At-

tributes that are semantically valid within the scenario but not part of the predefined expectations are excluded from the evaluation. This approach provides a practical and transparent way to assess the accuracy of the generated models.

Table 2: Example of test use case.

Name	Create Order
Primary User	Employee
Goal	To create a new order in the ERP system
Precondition	The employee is logged in the system
Postcondition	1. The new order was saved in the ERP System 2. A success message was displayed to the employee
Additional Users	None
Standard Procedure	1. The employee navigates to the 'Orders' page 2. The system shows a page containing a list of all orders and a button to create new orders 3. The employee clicks on the button for order creation 4. The system shows a form with fields for the customer and for one or more order positions with product, quantity and price each 5. The employee selects a customer by name or customer id, one or more products, sets quantity and price(s) and submits the form 6. The system calculates the total for the order, generates an order ID, saves the order in the database of the ERP system and displays a success message
Extensions	
Expected classes	Order, Position, Customer, Product
Expected attributes	• Order(id : String, total : double, customer : Customer, positions : Position[]) • Position(product : Product, quantity : int, price : double) • Customer(id : String, name : String)
Expected relationships	• Order 0..* ⟶ 1 Customer • Order 1 ⟶ 1..* Position • Position 0..* ⟶ 1 Product

5.1 Test Data Set

The test data set consists of eight different scenarios, each consisting of one or more use cases. For scenarios that contain more than one use case, a total list of expected classes, attributes, and relationships is provided; this can test the LLMs capability to identify overlaps between use cases taking place in the same domain. The test data set also contains suitable examples for specialization and inheritance, covering multiple types of relationships between classes. The list of test scenarios, along with the number of use cases and descriptions, can be found in Table 3.

Table 3: Test cases for benchmarking

Scenario	Use Cases	Description
Corporate	3	A generic corporate software with assignments between employees and customers and user management system
Customers	2	A webshop that serves both B2B and B2C customers, expecting specialization for a customer class
ERP	1	An ERP system in which orders can be created
Flight	1	A complex flight booking system with a user performing the check-in process
Healthcare	3	A healthcare software for a doctor's office with appointment management, with room for enums and inheritance
Stock	3	A stock exchange platform with portfolio and transaction management
Student	3	An educational software for student and course management
Webshop	3	A webshop including shopping, checkout and review processes

5.2 Accuracy Calculation

To calculate the accuracy of the proposed approach, the formulas for macro-average precision and recall are used:

$$\text{Macro-average Precision} = \frac{1}{N}\sum_{i=1}^{N}\frac{TP_i}{TP_i + FP_i}$$

$$\text{Macro-average Recall} = \frac{1}{N}\sum_{i=1}^{N}\frac{TP_i}{TP_i + FN_i}$$

where N represents the number of element types, TP_i the true positives, FP_i the false positives, and FN_i the false negatives for the i-th element type (Terven et al., 2025).

In the test data set, the number of expected and found elements is recorded for each element type. As previously stated, testable element types for domain class diagrams are classes, attributes, and relationships (i.e. $N = 3$). Based on this, let P be the macro-average precision in the context of a test scenario, which will be computed as:

$$P = \frac{1}{3}\cdot\left(\frac{TP_{Cls}}{TP_{Cls}+FP_{Cls}} + \frac{TP_{Attr}}{TP_{Attr}+FP_{Attr}} + \frac{TP_{Rel}}{TP_{Rel}+FP_{Rel}}\right)$$

and let R be the macro-average recall, that will be calculated as:

$$R = \frac{1}{3}\cdot\left(\frac{TP_{Cls}}{TP_{Cls}+FN_{Cls}} + \frac{TP_{Attr}}{TP_{Attr}+FN_{Attr}} + \frac{TP_{Rel}}{TP_{Rel}+FN_{Rel}}\right)$$

Using precision and recall values, we can now calculate its harmonic mean, also known as the F1 score,

which can be used as an indicator for the approach's performance (Terven et al., 2025):

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

5.3 Results

The results obtained from the developed approach directly address **RQ1**: Can LLMs generate domain class diagrams consisting of classes, attributes, and relationships from structured use case descriptions?

As demonstrated in Code Listing 5 and Figure 4, the proposed approach successfully generates domain class diagrams from structured use case inputs. The resulting diagrams include relevant classes, their corresponding attributes, and identified relationships with multiplicities. The generated PlantUML code illustrates how the structured output from the LLM can be automatically transformed into a valid visual representation, requiring no manual post-processing.

These results confirm that large language models can interpret structured use case descriptions effectively and produce domain class diagrams that align closely with expected outcomes, thereby supporting the feasibility of using LLMs in early stages of model-driven software development.

The second research question, **RQ2:** How complete are the generated domain class diagrams compared to manually designed domain class diagrams?, was investigated through a number of controlled test scenarios. A total of eight different scenarios (see Table 3) were created to represent a variety of software domains and complexity levels. Each scenario was executed five times to ensure the consistency of the results, leading to 40 analyzable test cases. The evaluation was conducted using both quantitative and qualitative methods.

To assess the completeness and correctness of the generated diagrams, three standard metrics were employed: precision, recall, and F1-score. Precision measures the proportion of correctly generated elements (classes, attributes, and relationships) among all elements produced by the model, reflecting its ability to avoid false positives. Recall, on the other hand, quantifies how many of the expected elements were correctly identified by the model, thus capturing its ability to avoid false negatives and produce complete diagrams. The F1-score, calculated as the harmonic mean of precision and recall, provides a balanced measure of overall accuracy, taking both correctness and completeness into account.

Overall, the system achieved stable accuracy across test runs. The macro-average precision ranged between 0.87 and 0.92, while recall values remained within 0.91 to 0.94, resulting in F1-scores between 0.89 and 0.93. These results indicate that the generated domain models are both largely accurate and complete when compared to manually designed diagrams, demonstrating that LLMs can capture essential structural elements from structured use case descriptions.

However, it is important to note that the evaluation was based on a relatively small and controlled dataset consisting of eight test scenarios. These scenarios were intentionally designed to cover common structures and relationships typically found in software domain models, which may have contributed to the consistently high precision, recall, and F1-scores. In more diverse or complex datasets—such as those featuring ambiguous requirements, overlapping entities, or unconventional relationship types—the models performance could vary significantly. For instance, higher variability in terminology and relationship semantics might reduce precision, as the model could generate redundant or contextually incorrect elements. Similarly, more complex scenarios may lower recall if certain subtle or implicit relationships are missed. Therefore, while the current results indicate strong performance within the test scope, further evaluation on larger and more heterogeneous datasets would be necessary to confirm the robustness and generalizability of the approach.

Simpler scenarios, such as Corporate and Customer, consistently produced very good results, likely due to their lower structural complexity (23 expected classes, 8 attributes, and 12 relationships). More complex cases, including ERP and Flight, revealed recurring issues. In the ERP scenario, multiplicities were often misrepresented—customers were linked to only one order, and products to a single order position—contradicting basic ERP logic (see Figure 5). Similarly, in the Flight scenario, incorrect multiplicities and one missing attribute were observed (see Figure 6). Specifically, a 1-to-1 relationship between Booking and Flight limited each flight to a single booking, and a many-to-1 relationship between Passenger and Seat allowed multiple passengers to share a seat. The Webshop scenario also showed missing attributes, such as the total cart amount and customer address fields in some runs.

As summarized in Table 4, the highest accuracy was achieved for classes and attributes, both with F1 scores of 0.97, while relationships showed lower performance with an F1 score of 0.77. The average overall F1 score across all element types was 0.91, with precision at 0.89 and recall at 0.93. The main source of error was incorrect multiplicities, suggesting that relationship identification remains a challenging task for LLMs.

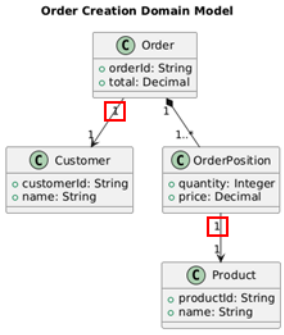

Figure 5: Problems with diagram for 'ERP' test scenario.

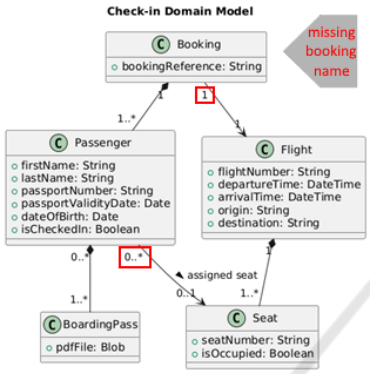

Figure 6: Problems with diagram for 'Flight' test scenario.

Table 4: Evaluation metrics by element type.

	Classes	Attributes	Relationships
Precision	0.99	1.00	0.67
Recall	0.94	0.94	0.91
F1 Score	0.97	0.97	0.77

These findings align with previous research, such as (Chen et al., 2023) and (De Bari et al., 2024), where LLMs performed best in identifying classes and attributes but struggled with relationship semantics. The results therefore confirm that, while the proposed approach can generate largely accurate domain models, further refinement is needed to improve relationship accuracy and multiplicity detection.

6 CONCLUSION & FUTURE WORK

This study presented an approach that leverages large language models (LLMs) to automatically generate domain class diagrams from structured use case descriptions. The method integrates natural language understanding with model generation logic to extract classes, attributes, and relationships, producing diagrams directly in PlantUML format. This automated approach minimizes manual modeling effort and demonstrates how LLMs can support early phases of model-driven software development. The approach was evaluated across eight controlled test scenarios representing different application domains and complexity levels. Simpler scenarios, such as Corporate and Customer, consistently yielded very good results due to their limited structural complexity. In contrast, more complex domains, such as ERP and Flight, exposed recurring issues, primarily related to incorrect multiplicities and missing attributes. For example, customers were sometimes linked to a single order, and products to one order position—both of which contradict standard ERP logic.

Quantitative results confirmed these tendencies: classes and attributes achieved high F1-scores of 0.97, whereas relationships scored lower at 0.77. The overall average F1-score across all element types was 0.91, with precision at 0.89 and recall at 0.93. These outcomes indicate that while LLMs can infer core domain structures, they still face challenges in accurately modeling relationships and multiplicities. This finding aligns with prior research (Chen et al., 2023; De Bari et al., 2024), where similar weaknesses were observed in relationship semantics.

It should be noted that the dataset used for evaluation was relatively small and well structured, which may have contributed to the high precision and recall observed. Broader and more diverse datasets, featuring more ambiguous or domain-specific use cases, could reveal further limitations in generalization. Future work should therefore focus on scaling the evaluation, improving prompt design, and integrating reasoning or validation mechanisms to enhance relationship accuracy.

Overall, the results demonstrate that the proposed LLM-based approach provides a promising foundation for semi-automated model generation in software engineering, minimizing the manual effort.

ACKNOWLEDGEMENTS

We acknowledge the use of ChatGPT to support language refinement and improve the clarity, readability of the manuscript. ChatGPT (OpenAI, GPT-5.2) is used as an editorial assistance tool and did not contribute to the design of the study, the analysis of results, or the scientific conclusions.

REFERENCES

Anda, B. and Sjøberg, D. I. K. (2005). Investigating the role of use cases in the construction of class diagrams. *Empir. Softw. Eng.*, 10(3):285–309.

Arulmohan, S., Meurs, M.-J., and Mosser, S. (2023). Extracting Domain Models from Textual Requirements in the Era of Large Language Models. In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pages 580–587.

Babaalla, Z., Jakimi, A., and Oualla, M. (2025). LLM-Driven MDA Pipeline for Generating UML Class Diagrams and Code. *IEEE Access*, 13:171266–171286.

Bruegge, B. and Dutoit, A. H. (2013). *Object-oriented software engineering using UML, patterns, and java: Pearson new international edition*. Pearson Education, London, England, 3 edition.

Cámara, J., Troya, J., Burgueño, L., and Vallecillo, A. (2023). On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML. *Software and Systems Modeling*, 22(3):781–793.

Chen, K., Yang, Y., Chen, B., Hernández López, J. A., Mussbacher, G., and Varró, D. (2023). Automated Domain Modeling with Large Language Models: A Comparative Study. In *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, pages 162–172. IEEE Press.

Cockburn, A. (1998). Basic use case template. *Humans and Technology, Technical Report*, 96.03a.

De Bari, D., Garaccione, G., Coppola, R., Torchiano, M., and Ardito, L. (2024). Evaluating Large Language Models in Exercises of UML Class Diagram Modeling. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ESEM '24, pages 393–399, New York, NY, USA. Association for Computing Machinery.

Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., and Zhang, J. M. (2023). Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53.

Ferrari, A., Abualhaija, S., and Arora, C. (2024). Model Generation with LLMs: From Requirements to UML Sequence Diagrams. *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*, pages 291–300.

Kokol, P. (2024). The Use of AI in Software Engineering: A Synthetic Knowledge Synthesis of the Recent Research Literature. *Information*, 15(6).

Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd Edition)*. Prentice Hall PTR, USA.

Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. (2022). Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097.

Li, Y., Keung, J., Ma, X., Chong, C. Y., Zhang, J., and Liao, Y. (2024). LLM-Based Class Diagram Derivation from User Stories with Chain-of-Thought Promptings. In *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 45–50.

Merrick, P. and Barrow, P. (2005). The Rationale for OO Associations in Use Case Modelling. *Journal of Object Technology*, 4(9):123–142.

PlantUML Team (2025a). Class Diagram. [Accessed 25-07-2025].

PlantUML Team (2025b). PlantUML. [Accessed 13-11-2025].

Schildmann, F. L. (2025). Automated UML Modeling:Generating Class Diagrams from Use Case Descriptions with LLMs. Bachelor thesis, Bielefeld University of Applied Sciences.

Shehata, M., Lepore, B., Cummings, H., and Parra, E. (2024). Creating uml class diagrams with general-purpose llms. In *2024 IEEE Working Conference on Software Visualization (VISSOFT)*, pages 157–158.

Shweta, Sanyal, R., and Ghoshal, B. (2021). Automated class diagram elicitation using intermediate use case template. *IET Software*, 15(1):2542.

Terven, J., Cordova-Esparza, D.-M., Romero-Gonzlez, J.-A., Ramrez-Pedraza, A., and Chvez Urbiola, E. (2025). A comprehensive survey of loss functions and metrics in deep learning. *Artificial Intelligence Review*, 58.

Ullrich, J., Koch, M., and Vogelsang, A. (2025). From Requirements to Code: Understanding Developer Practices in LLM-Assisted Software Engineering . In *2025 IEEE 33rd International Requirements Engineering Conference (RE)*, pages 257–266, Los Alamitos, CA, USA. IEEE Computer Society.

Wang, B., Wang, C., Liang, P., Li, B., and Zeng, C. (2024a). How LLMs Aid in UML Modeling: An Exploratory Study with Novice Analysts . In *2024 IEEE International Conference on Software Services Engineering (SSE)*, pages 249–257, Los Alamitos, CA, USA. IEEE Computer Society.

Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., and Wang, Q. (2024b). Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Trans. Softw. Eng.*, 50(4):911936.

Yang, Y., Chen, B., Chen, K., Mussbacher, G., and Varró, D. (2024). Multi-step Iterative Automated Domain Modeling with Large Language Models. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, MODELS Companion '24, page 587595, New York, NY, USA. Association for Computing Machinery.